

One-Bit Unlimited Sampling

Krunal Vaghela

May 2025

1 Problem Formulation

In practical hardware like analog-to-digital converter (ADC) constrained by hardware limitations, like finite dynamic range. The dynamic range defines the interval within which input signal amplitudes must lie in order to be accurately converted to digital signal. Signals exceeding this threshold result in clipping or saturation, leading to loss of information.

This limitation presents a fundamental challenge: how can we reliably store and later reconstruct signals whose amplitudes may exceed the dynamic range of the available ADC?

To address this, we do some sort of an encoding-decoding scheme. The key idea is to preprocess the signal before digitization, such that the transformed version lies within the ADC's dynamic range. This preprocessing must be reversible — allowing for accurate recovery of the original signal at the decoder, even though only the "compressed" or transformed version was stored or transmitted.

Hence, the central problem becomes:

- Design an encoding scheme that transforms a given signal with potentially high dynamic range into a bounded representation suitable for ADC.
- Develop a decoding algorithm that reconstructs the original signal from this encoded version with minimal error.

2 Unlimited Sampling via Modulo Quantization

Unlimited sampling, introduces an elegant workaround by employing a quantization scheme based on the modulo operation.

2.1 Modulo-Based Quantization

The core idea is to apply a **modulo operation** to the input signal before digitization, effectively “folding” it into a predefined range $[-\lambda, \lambda]$. This operation discards the most significant bits (MSBs) of the input value and retains only the least significant bits (LSBs).

Example: Consider a signal value of 15 and a modulo threshold $\lambda = 8$. The modulo-mapped value is:

$$15(mod)8 = 7$$

Here, 15 is the original (true) value, 8 is the folding threshold, and 7 is the retained part. This process is analogous to recording only the lower bits in a binary number while discarding the higher-order bits. In hardware terms, this allows the ADC to operate as if it only needs to measure values within $[-\lambda, \lambda]$, irrespective of the actual signal amplitude.

2.2 Trade-off Between Precision and Hardware Simplicity

Modulo quantization introduces a trade-off: it reduces the hardware complexity by limiting the required dynamic range, but at the cost of ambiguity in the measurement. Since the signal is folded, multiple values map to the same observed output.

However, this ambiguity can be resolved during reconstruction using oversampling. By sampling the signal at a rate significantly higher than the Nyquist rate — a technique referred to in the text as **reconstruction via constant-factor oversampling** — the decoder can leverage the redundancy to correctly unwrap the folded signal.

2.3 One-Bit Processing

A particularly interesting implication of modulo quantization is that in the extreme case only **one bit** (0 or 1, sort of signum function or comparator) may be needed to represent each sample.

3 Signal Assumptions and Superscillations

The unlimited sampling framework assumes that the input signal is **bandlimited** — meaning its frequency content is restricted to some maximum frequency Ω .

While reading the paper, I came across something that initially seemed confusing: the mention of **superscillatory functions**. These are functions that, despite being bandlimited, can locally oscillate faster than their highest frequency component. This felt paradoxical — how can something exceed its own frequency limit?

But as the paper explained, this is a known phenomenon, especially in areas like quantum physics. That made me feel more at ease — strange behavior is somewhat expected in the quantum world, and it's interesting that such functions can still appear within a bandlimited framework.

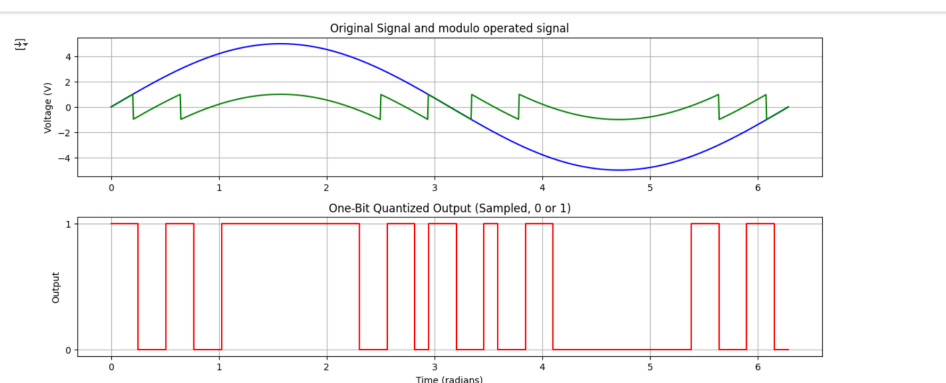


Figure 1: $\lambda=1$

the code for given image is available on the [Click here to open Colab notebook](#)

mention:1)knowledge about u2)difference bw figure of conventional samples and modulo samples3)about the convolutions involved,youngs inequality..be honest about few equations...inequality of delta g you have accepted it..

3.1 significance of u

Here u is some sort of noise term ..at every iteration in reconstruction noise term is updated in the sigma delta quantizer (SDQ) block.so variable can be defined as "noise-state of the block".

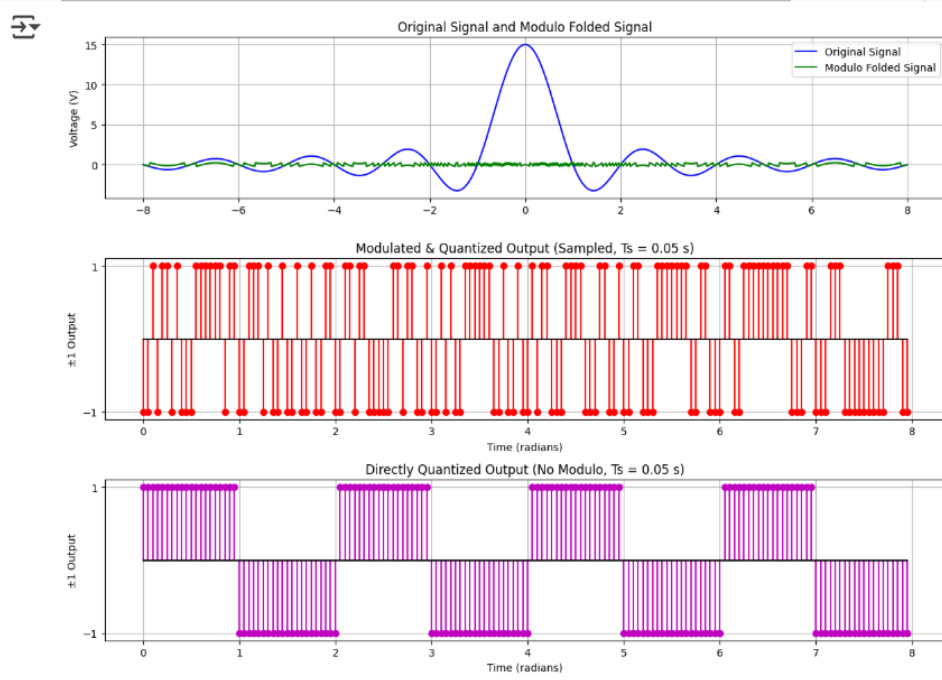


Figure 2: direct quantization vs modulated quantization

the code for given image is available on the [Click here to open Colab notebook](#)

the main benefit of one-bit modulo samples is that it capture sufficiently more information about the signal than conventional one-bit samples while Conventional one-bit samples exhibit saturation resulting from the fact that dynamic range exceeds $[-1,1]$. At every fluctuation from -1 to 1 we can interpret that signal has exceeded $n \cdot \lambda$ value.

4 Recovery algo

4.1 averaging samples

our main aim is to find the E_g term mentioned in the text. So first from the obtained modulated quantized samples we try to take average of samples, now why we do this? say any particular point has quantized output equal to 1 and its nearby points also have values equal to 1 hence instead of processing information about all neighbouring points we try to take average of all neighbouring points and show the averaged sum at that particular point. Now this can be done using low pass filters (this is a lot analogous to what I have done in image processing where low pass filters smoothen the image by sliding a 2-D kernel over the image). So in the text they have used B-spline functions. To my understanding B-splines of order 1 is basically a symmetric box function. order is convolution of 2 box functions which are triangular functions and by central limit theorem we can say B-spline of order infinity is a gaussian function. so convolute the b-spline function of order N with the quantized output to make computation of algo easier. Further they show how difference of zero order b-spline is equivalent to derivative of 1st order b-spline

4.2 high pass filters

now there can be instances when quantized jumps from -1 to 1 or vice-versa (obviously there are many!!) so corresponding to these sudden jumps we need to add high pass filters so we take finite difference of the calculated convolution (this is analogous to applying high pass filters on image to detect edges...edges can be thought of as sudden jumps in intensities)

4.3 significance of eq 8

the error in reconstructed signal can be thought of as sum of errors in quantized signal, E_g and error. Further we apply Young's inequality to derive the sufficiency condition on sampling period. The algorithm tries to find all the error terms to find $E_g[n]$ which is then further added to quantized signal to obtain a piecewise constant version of $g(t)$.

4.3.1 uncertainty in some equations

In some equations like eq 16, 17 I was unable to extract the intuition from them. Also if my understanding about some equations is way off then what should I be happy to get them rectified.